IEEESTEC – 19th Student Projects Conference, Niš, 2026.
https://doi.org/organizer will add number later
Original scientific paper

Fiducial: Keeping AI-Built Products Consistent

Aleksa Zdravković
12th grade high school student
ETŠ "Nikola Tesla" Niš
Niš, Serbia
aleksazdravkovic@proton.me
ORCID: 0009-0004-1535-1839

***Abstract*—This paper presents Fiducial, an open-source platform for building products that span software and hardware with artificial intelligence (AI) coding agents. Agents now write each part of a product quickly, but the facts the parts share are still copied between them and drift apart. Fiducial keeps each declared fact in one place, generates every artifact from it (code, translations, circuit board and enclosure) and checks every change, so drift between the parts is caught before a change is merged; whether a declared fact is itself correct remains for the person to judge. On 60 realistic mistakes in three sets, written by AI agents and saved before they were run, the checks caught 45 (75%; 95% interval 63–84%) on first run, where the same projects' ordinary continuous integration (CI) caught none. All but one of the 15 misses can be grouped into five classes of mistake; after one fix for each class, the checks catch 59 of the same 60, a measure of fit, not of generalization. Fiducial is open source.**

Keywords—AI agents, single source of truth, code generation, consistency checking, hardware–software co-design, continuous integration

I. Introduction

Building a product used to take a team: someone for the circuit board, someone for the firmware, others for the app and the enclosure. Artificial intelligence (AI) coding agents such as Claude Code [1] and Codex [2] are changing that: a firmware driver, a web page or a script that models a case can be written in minutes, so one person can now attempt a whole product.

Such a product is more than code. It has a circuit board and an enclosure, firmware, and an app or web page. It has text in every language it supports (translations, also called internationalization), a visual identity of colors, fonts and a logo, and a design system: the shared rules that keep its look the same and readable everywhere. Around these sit legal pages, search-engine metadata, a database schema and deployment settings. These parts look separate, but they share facts: a sensor reading's scale is in the firmware, the web page and the database, a brand color is in the page, the logo and the case, and a label must exist in every language.

This paper argues that there is a real advantage in centralizing all of this: the knowledge, the design system and the conventions in one place, and the work in one monorepository, a single repository that holds the code of many projects, as at Google [3]. Kept in a structured way, it also becomes reusable: a brand declared once supplies its colors, logo and 3D renderings wherever needed. Fiducial is built on this idea and is designed to cover all of the above.

Building such products, first Outreach [4], an off-grid messaging system of radios, gateways, an app and printed cases, and then the platform described here, has shown where the difficulty has moved. Writing each part is no longer the slow step; keeping the parts in agreement is. In Outreach [4], the firmware and the app disagreed on the layout of a message, and a printed case did not fit its board. Neither part was wrong on its own: a shared fact had been copied by hand, one copy changed, and the mistake appeared only when the parts met (Fig. 1a).

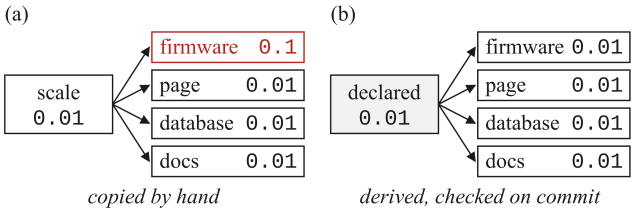


Fig. 1. One fact, the scale of a sensor reading, used in four places: (a) copied by hand, one copy changes and nothing notices; (b) declared once, every copy is generated from it and checked.

Agents make this problem faster. An agent sees only the code it works in: the one writing the firmware does not know the board changed, and the one writing the web page does not know the firmware did. Its output is usually right but never guaranteed, so more code written faster in more places gives an inconsistent product sooner.

What is missing is one shared and checked context for every discipline: a single place where a product's facts live, from which every part is produced, and which refuses a change that makes the parts disagree (Fig. 1b). This paper presents Fiducial [5], an open-source platform built to provide it. Its contributions are the declare, derive and check approach; an open-source implementation with 25 capabilities and a complete hardware and software example; and an evaluation on 60 mistakes written and saved before they were run, reported separately for first run and after fixes, against ordinary continuous integration (CI), the automatic build that checks every proposed change.

II. Existing Approaches

Several kinds of tool address parts of this problem, each keeping one kind of fact in agreement (Table I).

Model-driven and model-based systems engineering keep a design in one central model and generate or check other views from it [6], in languages such as the Systems Modeling Language (SysML) [7], with methods for managing inconsistencies between views [8], [9]. They are the closest in spirit, but the model lives in a specialized graphical tool, code is still written by hand beside it, and an agent cannot easily work in it. Engineering suites link two tools, such as a circuit board and its mechanical model [10]; the firmware and app stay outside.

Design-as-code tools write hardware as text (a circuit in atopile [11], a case in build123d [12]), which makes it writable by an agent, but each covers one discipline. Interface languages such as Protocol Buffers [13] declare a message once for both sides, which removes the drift met in Outreach, but only for messages. Other tools declare once for a single area: Zephyr's devicetree [14] and STM32CubeMX [15] for a microcontroller's hardware, OpenAPI [16] for an interface and Style Dictionary [17] for design tokens. Each generates parts of a product from one declaration, but we know of none that checks a whole product, hardware included.

Build systems such as Bazel [18] and Nix [19] rebuild exactly what changed. They make sure outputs follow from files; they do not check that files agree with each other, so the same fact typed into two files builds without complaint.

Coding agents follow context files such as AGENTS.md [20], which describe a repository's rules but do not enforce them, and their output is probabilistic and not guaranteed: a review of 72 studies finds functional bugs, including logic and semantic errors, the most common kind in AI-generated code [21]. A known answer is to pair a generator that can be wrong with a checker that cannot [22], but we know of no existing checker that spans a whole product.

TABLE I. EXISTING APPROACHES AND WHERE THEY STOP

Approach	**Keeps in agreement**	**Stops at**
Model-based engineering	views of one model	graphical tool; code by hand
Board–case links	board and case	two disciplines, two tools
Design as code	one discipline	facts shared with others
Interface languages	messages, both sides	everything else
Build systems, CI	outputs with files	two files that disagree
Fiducial	every declared fact	facts not yet declared

Fiducial keeps what works in each (text that agents can write, outputs tied to inputs by fingerprints, established engines for hard problems, and checks that decide) and changes the unit. It tracks facts rather than files or tools, across every discipline of one product, so any declared fact (a message, a pin, a color, a size) has exactly one home.

III. THE APPROACH

In one sentence: a fact is declared once, every part is generated from it, and a change that makes the parts disagree is caught before it is merged. Fiducial treats a product as facts and artifacts. Facts are the decisions that define it: which pin a sensor uses, what a button says in each language, the size of the board, the layout of a message. Artifacts are everything built from them: code, schematics, boards, translations, enclosure models. Drift is when an artifact stops agreeing with a fact, or with another artifact that uses it (Fig. 1a). Three rules keep drift out (Fig. 1b).

Declare once. Each declared fact is written in exactly one place, a declaration: a typed text file in the repository that a program can check. A fact that exists once has no second copy to disagree with.

Derive everything else. Every artifact that depends on a fact is generated from its declaration by one command and never edited by hand. Generation is meant to be deterministic, so any difference is meaningful: the placement search runs on one worker with a fixed seed and a pinned solver version. The autorouter is not, so the routed board is a build product judged by KiCad's design-rule check, not a hashed artifact.

Check every change. Before a change is accepted, checks confirm that every hashed artifact still matches what its declarations produce (the routed board is judged by KiCad's design-rule check) and that the declarations are valid; a failure names the declaration and the file at fault.

The rules check agreement, not truth. A fact declared wrongly (the wrong sensor address, a resistor of the wrong value) is copied faithfully into every artifact, and the product is then consistently wrong. Some such values break a rule and are refused, such as two colors with too little contrast, but whether a declaration is right is still decided by the person, the datasheet and the bench.

These rules change the role of an AI agent: it still edits declarations and code, but it is no longer trusted to keep anything consistent. Many of its mistakes are caught before they reach the product, with a message such as "field author is not in the collection's schema", meant to be precise enough to correct itself. Hence the name: a fiducial is a fixed reference point that everything is aligned to and compared against. The declarations are that reference for a product.

IV. IMPLEMENTATION

Fiducial is a command-line tool, fid, built by one person working with AI agents; Fig. 2 shows how a change moves through it.

A. One Structure for Every Product

fid new creates a product repository with a fixed layout, and fid add enables capabilities. A capability packages one discipline: its declarations, the generator that turns them into artifacts, its checks and instructions for agents. There are 25, from web applications to hardware. Every product has the same structure, so a new one starts from parts that already

work. Each also carries an AGENTS.md file for agents, and a hook stops an agent editing a generated file.

B. One Typed Language Underneath

The platform, and the code products share through it, is written in Rust, a compiled language with a strict type system: every mistake the compiler refuses is one the checks never have to catch, and its message names the file and line. Rust needs no runtime or garbage collector, so one body of code (message formats, identifiers and typed physical quantities) compiles into firmware, the fid tool and WebAssembly, which runs in the browser. The TypeScript types of the web code are generated from the Rust ones.

C. Declarations and Generation

Declarations are plain text files in version control, from the strings of each language to every part of a board with the net each pin connects to. Agents read and write text well, and changes go through code review. fid derive runs each capability's generator, from a typed module of translations to a firmware pin map or a KiCad circuit board [23]. A lock file records a cryptographic hash (a short fingerprint of a file's contents) for every input and every deterministic output.

D. Checks

Checks run in CI, and a change is merged only when they pass. There are three kinds. Freshness: fid derive --check compares every recorded fingerprint with the repository and fails if a declaration changed without its artifacts being regenerated, or if a generated file was edited by hand. Rules: generation refuses declarations that break a rule, such as a language missing a string, two colors with too little contrast, parts that cost more than the declared limit, or a board that does not fit its case. Consumers: compilers and type checkers read the generated files, so code that still uses a renamed fact stops compiling.

E. Engines for Hard Problems

Some generation steps are real engineering problems: where to place the parts on a board, how to connect them, how to build a case around them. The platform writes the problem down and hands it to an established engine: a constraint-programming solver, CP-SAT [24], finds a placement that satisfies the rules or proves none exists; build123d [12] builds the case; an autorouter routes the board and KiCad checks it. Every placement rule is one of eight general kinds, so a new product needs new declarations, not new code.

V. Example: A Sensor Stick

The repository contains a complete example: a USB-C stick that measures temperature and humidity and shows them on a web page (Fig. 3), with a printed case, a circuit board and Rust firmware. The person writes the declaration, the firmware's logic and the page; the routed board, schematic, bill of materials, case, manufacturing files and the code both sides use to exchange readings are generated. Changing the scale of a reading is one changed line: the check fails until the product is regenerated, after which the firmware's message code and the page's have both followed with no other edit.

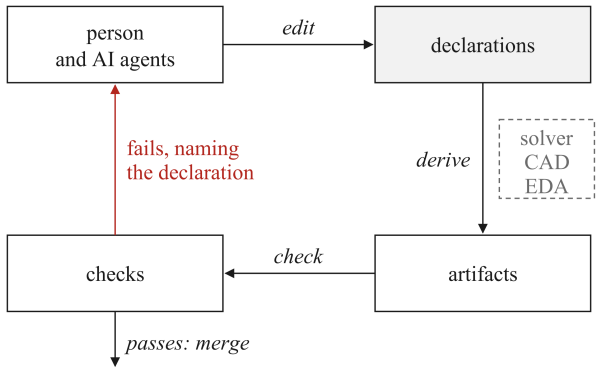


Fig. 2. How a change moves through Fiducial: declarations are edited, artifacts are generated with established engines, and the checks accept the change or name the declaration at fault.

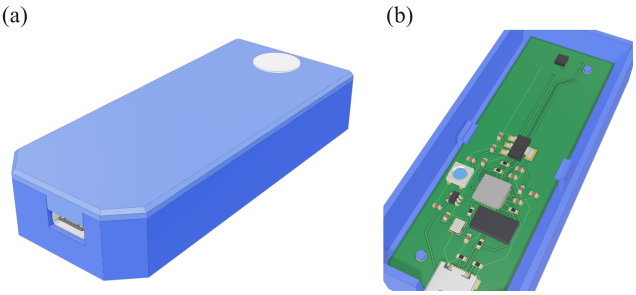


Fig. 3. The sensor stick, generated from its declaration: (a) the closed case; (b) the routed board inside it.

VI. Evaluation

To see whether the checks catch realistic mistakes, the test borrows mutation testing [25]: a realistic mistake is introduced on purpose, the checks are run, and the result shows whether they noticed. Each mistake was applied to a clean copy of one of two products, a fresh web product in English and Serbian or the sensor stick, and the checks ran as in CI. A mistake counts as caught if any check fails, whether or not its message names the right declaration. Of four sets of 20, the first covers eight kinds of drift met while building Fiducial and was used to develop it, so it is not an independent test and is left out of the counts. All 60 mistakes of the other three sets were written by AI agents and committed to the repository before their first run, so none could be chosen after seeing the result: the second set on 28 October 2026 by the AI that built Fiducial, the third on 28 October and the fourth on 1 November by separate agents that read only the two products, not the platform, the fourth aimed at the classes the third revealed. As a baseline, every set also ran through the projects' ordinary CI alone: builds, type checks and tests, without Fiducial's derive and check steps. That CI compares no artifact against another, so its 0 of 60 shows the gap Fiducial fills, not a flaw in those tools. Correct changes tested that the checks do not fail without a reason. Table II gives the results, with set 1, the development

set, shown but not counted; Fiducial 0.10.2 is the release after the fixes.

TABLE II. INJECTED MISTAKES CAUGHT

Mistakes	CI alone	First run	After fixes, same sets
Set 1, development	1/20	15/20	20/20
Set 2, by the AI	0/20	15/20	20/20
Set 3, by an agent	0/20	16/20	20/20
Set 4, by an agent	0/20	14/20	19/20
Correct changes	5/5	5/5	5/5

On first run the checks caught 45 of the 60 mistakes (75%, 95% Wilson interval 63–84%, treating mistakes as independent): 15, 16 and 14 of the three sets, which at 20 each are indicative only. Ordinary CI caught none of them, because nothing in it compares one artifact with another. All but one of the 15 misses can be grouped into five classes: a declared value typed again in code, such as a footer spelling out the domain; the check itself removed, or run after a step that regenerates what it checks; a declaration typed so loosely that a misspelled key or a missing entry is accepted; a rule from outside the product, such as a chip's flash size; and a generator reading a file nobody declared.

Each class now has one mechanism: a scan for declared values in hand-written code; a check that CI still runs the gate, and runs it first; typed declarations that reject unknown keys and require what a rule needs; outside rules kept as data, from datasheets, fabrication limits and law; and generators confined to their declared inputs. After these fixes, release 0.10.2 catches 59 of the same 60, but these sets produced the fixes, so this measures fit, not generalization. The one it misses is a mistake of meaning: contrast rules allow a lower ratio for large text, and a color pair was relabeled as large text. It passes because which text is large is not yet declared.

All four sets and their script are in the repository. Only first runs measure generalization, the fourth set was aimed at classes the third revealed, only two products have been built, and five correct-change runs are too few to show that the checks raise no false alarms. All 60 mistakes came from AI agents, possibly of the same model family as the AI that built Fiducial, so they may share its blind spots. The web product is online and tested; the sensor stick is designed and checked, but not yet manufactured and tested. The test measures the checks, not whether agents make fewer mistakes.

VII. CONCLUSION

AI agents make writing each part of a product fast, so the hard problem is keeping the parts in agreement. A probabilistic system cannot guarantee its output, and even a deterministic tool is only right about what it was told. Fiducial catches most inconsistency between the parts of a product, while whether each declared fact is true remains the person's to judge. On 60 mistakes it had not seen, its checks caught 45 (75%) on first run where ordinary CI caught none. All but one of the 15 misses can be grouped into five classes, each now addressed by one mechanism; on the same sets, which produced the fixes, 59 of the 60 are now caught, a measure of fit, and the last is a matter of meaning not yet declared. The next steps are to test it on mistakes written by other people, to apply the platform to new products, including systems of several controllers and a server, and to lower the tokens agents spend.

REFERENCES

[1] Anthropic, "Claude Code documentation," 2026. https://docs.anthropic.com/en/docs/claude-code

[2] OpenAI Codex. https://openai.com/codex/

[3] R. Potvin and J. Levenberg, "Why Google stores billions of lines of code in a single repository," Commun. ACM, vol. 59, pp. 78–87, 2016.

[4] A. Zdravković, Đ. Jovanović and P. Šubarević, "Outreach: Modular AI-enhanced off-grid mesh communication system for remote environments," IEEESTEC – 18th Student Projects Conference, Niš, 2025.

[5] A. Zdravković, Fiducial, release 0.10.2, 2026. https://github.com/AleksaZCodes/fiducial

[6] D. C. Schmidt, "Guest editor's introduction: Model-driven engineering," Computer, vol. 39, no. 2, pp. 25–31, 2006, doi: 10.1109/MC.2006.58.

[7] Object Management Group, OMG Systems Modeling Language (SysML), version 2.0, Sept. 2025. https://www.omg.org/spec/SysML/2.0

[8] B. Nuseibeh, S. Easterbrook and A. Russo, "Leveraging inconsistency in software development," Computer, vol. 33, no. 4, pp. 24–29, 2000, doi: 10.1109/2.839317.

[9] S. J. I. Herzig, A. Qamar and C. J. J. Paredis, "An approach to identifying inconsistencies in model-based systems engineering," Procedia Computer Science, vol. 28, pp. 354–362, 2014.

[10] Altium, MCAD CoDesigner technical documentation. https://www.altium.com/documentation/altium-codesigner

[11] atopile, a language for designing circuit boards as code. https://github.com/atopile/atopile

[12] build123d, a Python CAD library on the OpenCascade kernel. https://github.com/gumyr/build123d

[13] Protocol Buffers. https://protobuf.dev/

[14] Zephyr Project, Devicetree. https://docs.zephyrproject.org/latest/build/dts/index.html

[15] STMicroelectronics, STM32CubeMX. https://www.st.com/en/development-tools/stm32cubemx.html

[16] OpenAPI Initiative, OpenAPI Specification. https://spec.openapis.org/oas/latest.html

[17] Style Dictionary. https://github.com/style-dictionary/style-dictionary

[18] Bazel, a build and test tool. https://bazel.build/

[19] E. Dolstra, M. de Jonge and E. Visser, "Nix: A safe and policy-free system for software deployment," in Proc. LISA, 2004, pp. 79–92.

[20] AGENTS.md, a format for guiding coding agents. https://agents.md/

[21] R. Gao, A. Tahir, P. Liang, T. Susnjak and F. Khomh, "A survey of bugs in AI-generated code," arXiv:2512.05239, 2025.

[22] S. Kambhampati et al., "Position: LLMs can't plan, but can help planning in LLM-Modulo frameworks," in Proc. ICML, PMLR 235, 2024.

[23] KiCad EDA. https://www.kicad.org/

[24] L. Perron and F. Didier, CP-SAT, Google OR-Tools. https://developers.google.com/optimization/cp/cp_solver

[25] R. A. DeMillo, R. J. Lipton and F. G. Sayward, "Hints on test data selection: Help for the practicing programmer," Computer, vol. 11, no. 4, pp. 34–41, 1978, doi: 10.1109/C-M.1978.218136.